

Learning When Not to Attend Globally

Anonymous ACL submission

Abstract

When reading books, humans focus primarily on the current page, revisiting earlier pages only when necessary. Similarly, we show that LLMs can learn to decide when global context is needed. We propose All-or-Here Attention (AHA), which uses a binary router for each attention head to switch between full attention (All) and local sliding-window attention (Here) on a per-token basis. By decoupling local processing from global access, AHA shows that full attention is largely redundant, and that efficient inference requires only on-demand access to global context. Experiments show that AHA can replace up to 88% of full attention operations with sliding-window attention on standard benchmarks without losing performance. Furthermore, AHA is complementary to various existing sparse attention frameworks. For instance, when integrated with DuoAttention, it further reduces global attention overhead while maintaining comparable accuracy on long-context tasks, unlocking additional efficiency beyond static allocation. Our model will be made publicly available.

1 Introduction

The self-attention mechanism (Bahdanau et al., 2014) serves as the cornerstone of the Transformer architecture (Devlin et al., 2019; Brown et al., 2020; Vaswani et al., 2017), driving the success of modern Large Language Models (Ouyang et al., 2022; Touvron et al., 2023; Yang et al., 2025a). However, its quadratic complexity with respect to sequence length poses significant computational challenges. To address this, numerous studies have proposed efficiency techniques, including linear attention (Katharopoulos et al., 2020; Gu and Dao, 2024; Yang et al., 2024), sparse attention (Child et al., 2019a; Beltagy et al., 2020a; Zaheer et al., 2020), and hierarchical attention (Yang et al., 2016; Yuan et al., 2025). While these methods often focus on approximating full attention or compressing

state representations, we suggest that the intrinsic redundancy of the attention mechanism enables a far simpler solution.

In this paper, we propose a straightforward paradigm: enabling the Transformer to *learn when not to attend globally*. Our design is motivated by the observation that tokens exhibit heterogeneous context requirements. Intuitively, maintaining syntactic continuity and local coherence should primarily require immediate context, whereas global retrieval is necessary only for a sparse subset of tokens resolving long-range dependencies (Liu et al., 2017; Fan and Jiang, 2023; Futrell et al., 2015; Beltagy et al., 2020a). Figure 1 shows a concrete example. Based on this perspective, we introduce **All-or-Here Attention (AHA)**. By employing a conditional hard gating mechanism, AHA dynamically toggles between full attention (All) and local sliding window attention (Here) for each attention head.

Specifically, we integrate lightweight routers within each Transformer layer to generate scalar importance scores. Based on these scores and a pre-defined threshold, each attention head dynamically switches between full and sliding-window attention on a per-token basis. Rather than diverging from existing sparse attention paradigms, AHA introduces an orthogonal dimension to the design space. For instance, static head-partitioning methods like DuoAttention (Xiao et al., 2025) and Prulong (Bhaskar et al., 2025) explore *which* head should attend globally, while dynamic sparse methods like NSA (Yuan et al., 2025) and Quest (Tang et al., 2024) design sophisticated mechanisms to determine *how* to attend globally by selectively indexing historical keys. In contrast, AHA answers a more primitive question: *when* (not) to attend globally. We show that a binary "All-or-Here" selection is sufficient to unlock massive sparsity, suggesting that capturing the "when" is perhaps the most fundamental factor in efficient attention modeling.

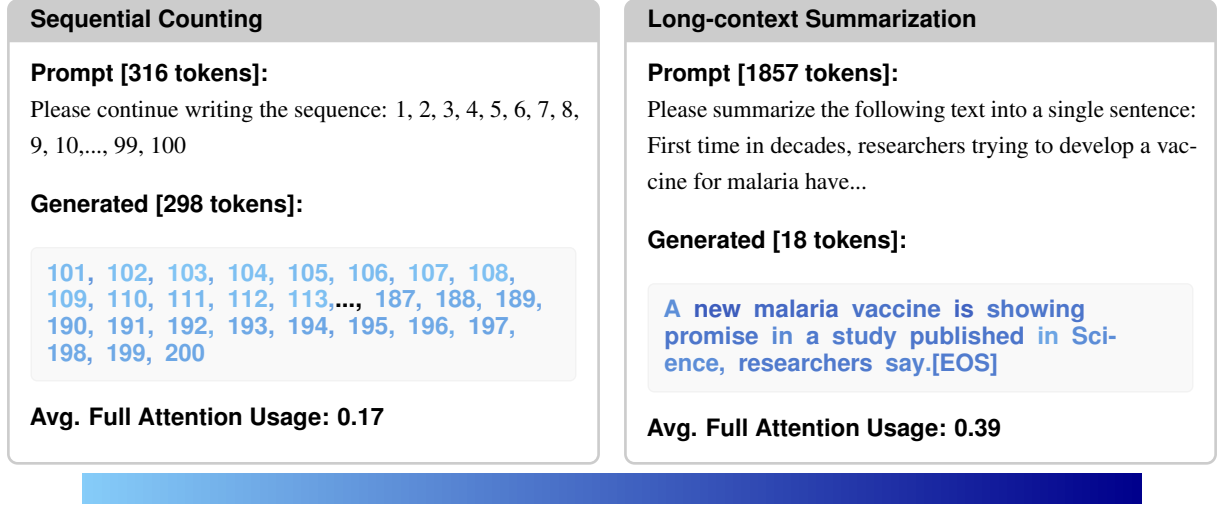


Figure 1: Visualization of average full attention usage across tasks (local attention window size, $w = 128$). The color gradient indicates the ratio of activating full attention, where dark blue represents 1.0 (Full Attention) and light blue represents 0.0 (Local Attention). This shows that reliance on global context varies by task and token.

Moreover, AHA is inherently composable with many existing methods. For instance, when integrated with DuoAttention (Xiao et al., 2025), AHA can be applied within its static retrieval heads to enable dynamic, token-level routing. Similarly, for selective attention methods like NSA (Yuan et al., 2025) and Quest (Tang et al., 2024), their fine-grained indexing mechanisms are executed only when AHA decides to trigger global attention; otherwise, tokens strictly utilize the simple sliding-window branch. By relying on a hard decision at the routing stage rather than a soft feature fusion of multi-granularity contexts, this operational separation allows AHA to serve as a clean, front-end filter that enhances diverse attention architectures without interfering with their internal mechanisms.

Experiments demonstrate that the vast majority of full attention operations in pre-trained LLMs can be safely localized. Specifically, by applying AHA to OLMo-2-1B (OLMo et al., 2024) with a local window size of 128, we show that over 88% of full attention computations can be substituted by sliding window attention while preserving full performance on standard benchmarks. Furthermore, to demonstrate its composability with existing frameworks, we integrate AHA with DuoAttention (Xiao et al., 2025) by applying dynamic routing exclusively to its static retrieval heads, using Llama-3-8B-Instruct (Grattafiori et al., 2024) as the backbone. On the context-intensive LongBench suite (Bai et al., 2024), this combined architecture successfully eliminates over 65% of full attention operations with minimal performance degradation,

yielding up to a 2.29x decoding speedup over the full attention baseline in long-context scenarios. While AHA can naturally be integrated with other sparse attention frameworks, we leave these explorations to future work. Ultimately, these findings suggest that beyond optimizing the global attention mechanism itself, a significant opportunity for attention optimization lies simply in accurately identifying *when* global context is actually required.

2 Method

The core intuition behind AHA is to assign a binary gate to each attention head, dynamically routing tokens to either a full ('all') or a local sliding window ('here') attention path. Although conditional computation (Bengio et al., 2015; Bapna et al., 2020) and its application to attention mechanisms (Ainslie et al., 2023; Zhang et al., 2022) have been discussed in prior works, here we investigate the implications of enabling such dynamic sparsity within pre-trained decoder-only language models to characterize their intrinsic attention requirements.

2.1 Architecture

Figure 2 illustrates the pipeline of a single All-or-here Attention Block. The core structure is a dynamic routing mechanism that adaptively toggles between full and local attention.

Let the input hidden states be $X \in \mathbb{R}^{n \times d}$, where n is the sequence length and d is the model dimension. Each Transformer block comprises m attention heads, denoted by the set $H =$

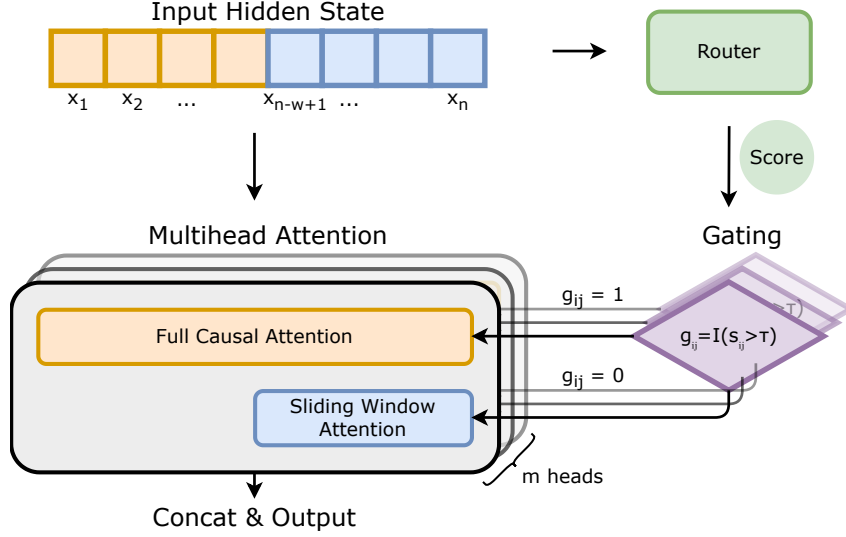


Figure 2: Overview of the All-or-Here Attention (AHA) architecture. A lightweight router computes importance scores for each head, generating binary gates that dynamically toggle between full and local attention.

$\{\text{head}_1, \dots, \text{head}_m\}$. To achieve input-adaptive routing, we employ a lightweight router $W_{\text{Router}} \in \mathbb{R}^{d \times m}$ that generates importance scores dynamically. The resulting score matrix $S(X) \in \mathbb{R}^{n \times m}$ is conditioned on the input X and computed as:

$$S(X) = \sigma(XW_{\text{Router}}). \quad (1)$$

where the σ represents the sigmoid activation. This formulation ensures that the attention scope for each token is not statically assigned but rather determined by the specific contextual information in X . This represents a major departure from methods like DuoAttention and Prulong, which rely on fixed, pre-defined functional roles for each head.

The element $s_{i,j} \in (0, 1)$ of the matrix $S(X)$ represents the importance score for the i -th token within the j -th attention head. This score $s_{i,j}$ governs the binary routing decision for each specific token-head pair via a thresholding mechanism:

$$g_{i,j} = \mathbb{I}(s_{i,j} > \tau), \quad (2)$$

where $\mathbb{I}(\cdot)$ is the indicator function and τ is a pre-defined threshold. The binary gate $g_{i,j}$ determines the attention granularity: $g_{i,j} = 1$ triggers full causal attention, whereas $g_{i,j} = 0$ restricts the computation to local sliding window attention. To optimize the non-differentiable indicator function, we employ the Straight-Through Estimator (STE) (Bengio et al., 2013). While the discrete gate $g_{i,j}$ determines the attention path during the forward pass, the backward pass approximates the thresholding as an identity function. This bypasses

the zero-gradient bottleneck of discrete operations, enabling gradient propagation through the hard gating mechanism.

With the binary gate $g_{i,j}$ determined, the model conditionally executes the attention mechanism. Let w denote the pre-defined size of the local sliding window. When the gate is activated ($g_{i,j} = 1$), the attention output $\mathbf{a}_{i,j}$ for the j -th head at token step i is computed using the entire prefix (Full Attention):

$$\mathbf{a}_{i,j} = g_{i,j} \cdot \text{Attn}(\mathbf{q}_{i,j}, \mathbf{K}_{:,i,j}, \mathbf{V}_{:,i,j}). \quad (3)$$

Conversely, when $g_{i,j} = 0$, the model restricts its context to a local sliding window of the most recent w tokens:

$$\mathbf{a}_{i,j} = (1 - g_{i,j}) \cdot \text{Attn}(\mathbf{q}_{i,j}, \mathbf{K}_{i-w:i,j}, \mathbf{V}_{i-w:i,j}), \quad (4)$$

where $\mathbf{q}_{i,j}$ is the query vector for the current token. The notation $\mathbf{K}_{:,i,j}$ represents the key states for the entire prefix, whereas $\mathbf{K}_{i-w:i,j}$ denotes the truncated key states restricted to the local window.

Following the conditional computation, we aggregate the results to form the final block output. For the current token i , the outputs from all m heads are concatenated and projected via the output weight matrix $\mathbf{W}_O \in \mathbb{R}^{d \times d}$:

$$\mathbf{O}_i = \text{Concat}(\mathbf{a}_{i,1}, \dots, \mathbf{a}_{i,m})\mathbf{W}_O. \quad (5)$$

Beyond the specific modifications to the attention mechanism, our architecture strictly adheres to the standard decoder-only Transformer design (Brown et al., 2020; Touvron et al., 2023).

2.2 Loss Function

To balance language modeling performance with attention sparsity, we optimize the model using a joint objective function:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \mathcal{L}_{\text{reg}}, \quad (6)$$

where $\mathcal{L}_{\text{LM}}$ is the standard next-token cross-entropy loss and λ controls the sparsity trade-off. Specifically, $\mathcal{L}_{\text{reg}}$ applies an L_1 penalty to the router scores across all L layers, n tokens, and m heads:

$$\mathcal{L}_{\text{reg}} = \frac{1}{L \cdot n \cdot m} \sum_{k=1}^L \sum_{i=1}^n \sum_{j=1}^m s_{i,j}^{(k)}. \quad (7)$$

Minimizing $\mathcal{L}_{\text{reg}}$ drives the scores $s_{i,j}^{(k)}$ toward zero, thereby lowering the frequency of triggering full attention via the gating mechanism $g_{i,j} = \mathbb{I}(s_{i,j} > \tau)$. This incentivizes the model to default to efficient sliding window attention, invoking global context only when necessary.

3 Experiments

3.1 Implementation Details

To evaluate the efficacy of our proposed method, we implement the AHA architecture using OLMo-2-0425-1B-SFT (OLMo et al., 2024) as the base model, whose training data is entirely open-source. We initialize the model with all original pre-trained parameters. The gating mechanism operates with a fixed threshold of $\tau = 0.5$. Regarding the training objective, we set the regularization coefficient to $\lambda = 3 \times 10^{-4}$. We fine-tune the model for one epoch on the Tulu-v3 dataset (Lambert et al., 2024), which is the same dataset used to train the base model. Optimization is performed using AdamW with a learning rate of 3×10^{-5} , $\beta_1 = 0.9$, and $\beta_2 = 0.95$, alongside a linear warmup ratio of 0.03 and a global batch size of 128.

For comparison, we also implement AHA on top of DuoAttention (Xiao et al., 2025), using Llama-3-8B-Instruct (Grattafiori et al., 2024) as the base model. Specifically, we select DuoAttention’s configuration with a 50% static retrieval head split as our baseline, and apply AHA’s dynamic routing exclusively to these designated retrieval heads. For a fair comparison, we align our architectural constraints with DuoAttention by freezing all original model weights and training only the lightweight gating routers. We configure the local sliding window to utilize 128 attention sinks and 256 recent

tokens, which is the same as DuoAttention. Since this parameter-efficient setup focuses solely on optimization of the routers, we apply a different regularization penalty of $\lambda = 5 \times 10^{-2}$ and a learning rate of 3×10^{-4} . The model is trained on the BookSum dataset (Kryściński et al., 2022), while all other optimization hyperparameters remain consistent with the OLMo-2 experiment.

3.2 Benchmarks

We evaluate our method across six benchmarks using the lm-evaluation-harness toolkit (Gao et al., 2023): MMLU (Hendrycks et al., 2021), HellaSwag (Zellers et al., 2019), CSQA (Talmor et al., 2019), GSM8K (Cobbe et al., 2021), MBPP (Austin et al., 2021), and MultiNews (Fabbri et al., 2019). We employ a 5-shot setting for MMLU, HellaSwag, CSQA, and GSM8K, and a zero-shot setting for MBPP and MultiNews. Evaluation metrics include accuracy (Acc) for MMLU and CSQA, normalized accuracy (Acc_norm) for HellaSwag, exact-match (EM) for GSM8K, Pass@1 for MBPP, and ROUGE scores for MultiNews. To ensure the sliding window attention is strictly operative, we filter out samples where the input context length is shorter than the window size. The average input context lengths for these tasks are 755, 532, 332, 939, 675, and 2,196 tokens, respectively.

Furthermore, to evaluate model performance under demanding, long-context scenarios, we employ the LongBench suite (Bai et al., 2024), which encompasses various task types including Multi-Doc QA, Single-Doc QA, Summarization, Few-shot Learning, Synthetic Tasks, and Code Completion. The average input context length for LongBench is 8,071 tokens, far exceeding the 256-token sliding window size configured for our long-context evaluations.

3.3 Main Results

We evaluate the proposed attention mechanism by training and testing a suite of models with varying local sliding window sizes $w \in \{16, 32, 64, 128\}$. This setup allows us to investigate how the capacity of the local context influences the router’s reliance on full attention. Table 1 summarizes the performance of the proposed All-or-here Attention across various window sizes. We observe a clear positive correlation between the window size and model performance. As the window size w increases, the model’s accuracy progressively re-

Method	MMLU	HellaSwag	CSQA	GSM8K	MBPP	News	Retain %
Vanilla	0.4056	0.6487	0.5307	0.3874	0.1560	0.2074	100.0%
$w = 16$	0.3676	0.6400	0.4578	0.3980	0.1260	0.2014	92.7%
$w = 32$	0.3847	0.6410	0.4586	0.4102	0.1240	0.1814	92.2%
$w = 64$	0.3963	0.6431	0.4980	0.4121	0.1200	0.1983	94.9%
$w = 128$	0.4087	0.6445	0.4996	0.4291	0.1580	0.2053	100.9%

Table 1: Performance comparison of the proposed All-or-Here Attention across various window sizes. "Vanilla" denotes the OLMo-2-0425-1B-SFT model using full attention.

Method	MMLU	HellaSwag	CSQA	GSM8K	MBPP	News	Avg %
Vanilla	100.0%	100.0%	100%	100%	100%	100%	100.0%
$w = 16$	66.0%	71.8%	59.5%	41.5%	41.3%	36.0%	52.7%
$w = 32$	52.1%	58.0%	43.7%	35.2%	32.0%	27.1%	41.4%
$w = 64$	34.4%	38.3%	24.9%	24.0%	24.6%	22.5%	28.1%
$w = 128$	11.2%	11.8%	7.4%	11.3%	13.6%	14.4%	11.6%

Table 2: Average Full Attention usage of the proposed All-or-here Attention across various window sizes.

covers, eventually matching the vanilla result of OLMo-2-SFT. Specifically, with window sizes of $w = 128$, AHA maintains 100.9% of the Vanilla performance. Even under the extreme constraint of $w = 16$, the model retains 92.7% of the original performance.

Table 2 details the sparsity patterns across different benchmarks. We define the average full attention usage μ_f as the mean of the binary gating values $g_{i,j}^{(k)}$ (Equation 2) averaged across all L layers, m heads, and n generated tokens:

$$\mu_f = \frac{1}{L \cdot n \cdot m} \sum_{k=1}^L \sum_{i=1}^n \sum_{j=1}^m g_{i,j}^{(k)}. \quad (8)$$

This metric directly reflects the frequency with which the model selects the full attention path. At $w = 128$, the model successfully bypasses 88.4% of the full attention operations. Our method also exhibits robust adaptability to the available window size. Moreover, we observe that the demand for full attention diminishes rapidly as the local window expands. This empirical evidence suggests that the vast majority of tokens depend primarily on the local area, while only a sparse minority necessitate attention to access the global context.

As shown in Figure 3, we rank all 256 attention heads ($L = 16, m = 16$) of the trained $w = 128$ AHA model by their average full attention usage μ_f . In this plot, the x-axis represents the individual attention heads sorted in descending order of their usage frequency, while the y-axis denotes the corresponding average full attention usage score (μ_f). The figure clearly illustrates a pronounced long-tail

distribution that remains highly consistent across all evaluated tasks. A small fraction of heads exhibits high utilization of full attention, whereas the vast majority constitutes a long tail with nearly zero μ_f , indicating that they operate almost exclusively within the local sliding window. This structural long-tail distribution provides a possible explanation for the efficacy of static head-allocation methods like DuoAttention (Xiao et al., 2025); such approaches succeed by permanently assigning local attention to the long-tail heads and full attention to the high-usage heads. By dynamically regulating global retrieval on a per-token basis, AHA reduces redundant full-attention computations even within the most globally dependent heads.

3.4 Long-context performance comparison

To evaluate the scalability and effectiveness of our method in demanding long-context scenarios, we conduct comprehensive experiments using the Llama-3-8B-Instruct model (Grattafiori et al., 2024) as the backbone. Specifically, DuoAttention relies on a static pre-classification of attention heads into streaming heads and retrieval heads. For our AHA setup, we freeze the backbone and apply token-level dynamic gates exclusively on top of DuoAttention’s static retrieval heads. By applying varying regularization penalties during training, we plot the Pareto frontier of average Long-Bench scores against full-attention sparsity (Figure 4). The results demonstrate that at an equivalent accuracy level, AHA successfully extends the sparsity frontier of DuoAttention, yielding a superior Pareto curve.

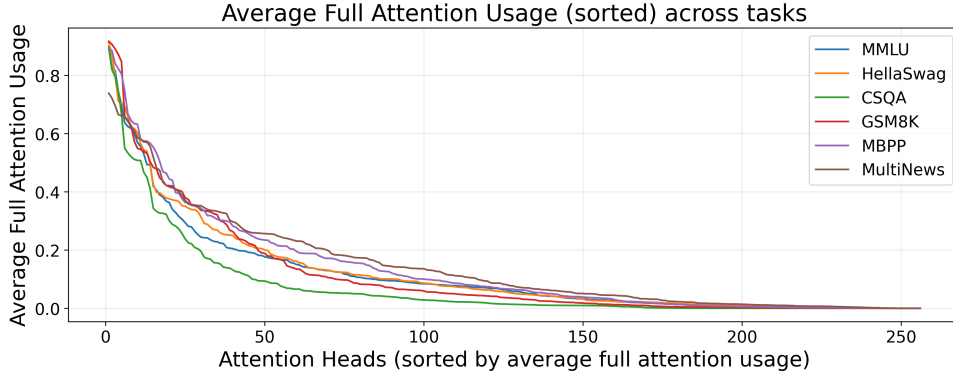


Figure 3: Sorted average full attention usage for all attention heads. The data shows that a small minority of heads account for the majority of full attention usage.

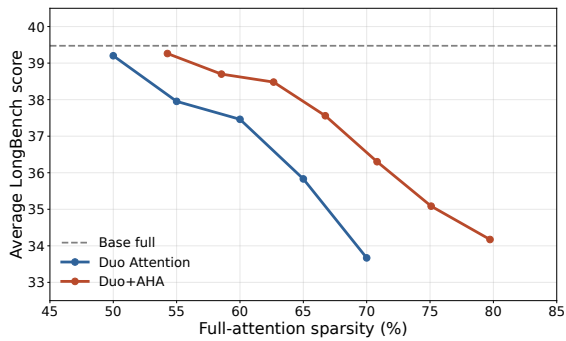


Figure 4: Pareto frontier of Average LongBench score versus full-attention sparsity (%). AHA extends the sparsity frontier of the DuoAttention baseline.

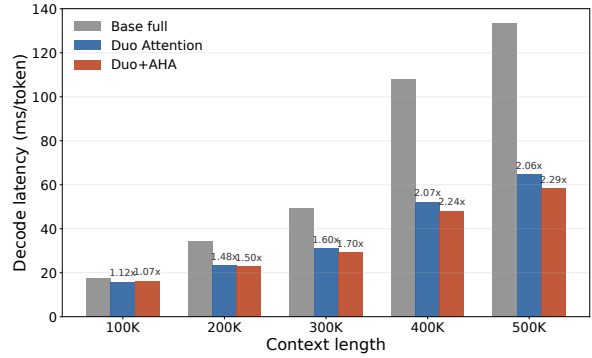


Figure 5: Decode latency (ms/token) comparison across varying context lengths. Measurements are conducted on a single H100 GPU utilizing the Lserve engine.

To evaluate the impact of these sparsity reductions on actual inference speed, we measure the decode latency across varying context lengths. For this evaluation, we specifically select the configurations of DuoAttention and Duo+AHA from Figure 4 that achieve a comparable average LongBench score of approximately 37.5. Measurements are conducted using a batch size of 1 on a single NVIDIA H100 GPU, with both configurations deployed on the Lserve inference engine (Yang et al., 2025b). As shown in Figure 5, at longer context lengths such as 400K and 500K tokens, our approach further accelerates the already-sparsified DuoAttention by approximately 10%, pushing the overall speedup from 2.06x to 2.29x at 500K tokens compared to the full attention baseline. However, the additional latency reduction achieved by AHA remains relatively modest. We hypothesize that this stems from our parameter-efficient training setting. Specifically, the minimal number of trainable parameters and the SFT-only training likely constraint the model from developing a deeper understanding of the context required for optimal routing deci-

sions. Therefore, it is worth exploring whether incorporating such dynamic gating mechanisms directly during the pre-training phase could better capture these contextual dependencies, potentially translating into further latency reductions.

3.5 Ablation Studies

Impact of Regularization Penalty. To investigate the trade-off between efficiency and performance, we trained models with varying L_1 regularization coefficients $\lambda \in \{1 \times 10^{-4}, 3 \times 10^{-4}, 1 \times 10^{-3}\}$ while fixing the window size at $w = 128$. Table 3 summarizes the results. We observe that the penalty coefficient serves as a pivotal hyperparameter for balancing sparsity and model capability. An overly aggressive penalty ($\lambda = 1 \times 10^{-3}$) forces the model into extreme sparsity (only 4.5% global attention), precipitating performance degradation, particularly on reasoning-intensive tasks like GSM8K (dropping to 0.3730) and MBPP (dropping to 0.1320). This indicates that complex reasoning necessitates a minimum threshold of global context. Conversely, relaxing the penalty

Penalty (λ)	MMLU	HellaSwag	CSQA	GSM8K	MBPP	News	Avg Full %
1×10^{-3}	0.4045	0.6419	0.4878	0.3730	0.1320	0.1997	4.5%
3×10^{-4}	0.4087	0.6445	0.4996	0.4291	0.1580	0.2053	11.6%
1×10^{-4}	0.4042	0.6456	0.4996	0.4397	0.1600	0.2047	53.5%

Table 3: Ablation study on the L1 regularization penalty (λ).

Setting	MMLU	HellaSwag	CSQA	GSM8K	MBPP	News	Avg Full %
Top-32	0.3962	0.6399	0.4976	0.0682	0.1180	0.1973	12.5%
Top-64	0.3938	0.6349	0.5078	0.1562	0.1340	0.1958	25.0%
Top-128	0.3962	0.6399	0.5291	0.4064	0.1540	0.2031	50.0%
Top-256 (full)	0.3975	0.6419	0.5127	0.4238	0.1640	0.2047	100.0%

Table 4: Ablation study on static head assignment. Heads are ranked by their average full attention usage. "Top- k " indicates the number of heads assigned to full attention.

to $\lambda = 1 \times 10^{-4}$ significantly increases global attention usage to 53.5% without yielding meaningful performance gains. This suggests that even with a larger computational budget for global attention, the model extracts no additional signal, confirming that full attention is largely redundant.

Dynamic vs. Static Head Assignment. To verify the necessity of the dynamic routing mechanism, we compare AHA against a static assignment baseline. As shown in Figure 3, we rank all 256 attention heads ($L = 16, m = 16$) of the trained AHA model by their average full attention usage μ_f . We then evaluate static variants where the top- k heads are fixed to full attention while the remaining heads are restricted to a local sliding window of $w = 128$.

Table 4 presents the results for $k \in \{32, 64, 128, 256\}$. We observe that static assignment is significantly less efficient than AHA’s adaptive routing. For instance, the Top-32 static configuration utilizes 12.5% full attention, yet its performance is markedly inferior to the default AHA ($w = 128$), which achieves better results across nearly all benchmarks while using only 11.6% full attention (as shown in Table 3). The disparity is particularly acute in reasoning-heavy tasks; on GSM8K, the Top-32 static model drops to a near-zero score of 0.0682, whereas AHA maintains 0.4291. Even as we increase the static budget to Top-128 (50% full attention), the performance on GSM8K and MBPP still lags behind our adaptive model. These results demonstrate that the optimal attention scope for a head is not merely a function of its identity but is highly dependent on the real-time context. AHA’s ability to dynamically evoke full attention on demand allows it to preserve critical reasoning capabilities with a smaller

computational footprint.

4 Related Work

4.1 Efficient Attention

To address the quadratic computational complexity of standard self-attention, efficient attention mechanisms are generally categorized into linear, sparse, and hierarchical approaches. Linear attention reformulates the attention mechanism to scale linearly with sequence length, often employing kernel feature maps or recurrent state updates, as seen in the autoregressive framework of LinearAttn (Katharopoulos et al., 2020) and the selective state-space models of Mamba (Gu and Dao, 2024). Sparse attention reduces memory footprint by restricting token interactions to specific subsets. This includes static block-sparse patterns in Sparse Transformers (Child et al., 2019a) and structured sliding windows in Longformer (Beltagy et al., 2020a) and BigBird (Zaheer et al., 2020). More advanced works have explored adaptive sparsity, ranging from Differentiable Attention Window (Nguyen et al., 2020) to Adaptive Attention Span (Sukhbaatar et al., 2019; Fu et al., 2025), which dynamically determine the optimal context span for each head. Finally, hierarchical attention organizes information processing across multiple levels of granularity, from the sentence-to-document aggregation in HAN (Yang et al., 2016) to the hardware-aligned, coarse-to-fine compression in Native Sparse Attention (Yuan et al., 2025).

In contrast to these increasingly sophisticated designs, our method demonstrates that complex span decision processes or hierarchical selection could be simplified when handling many tasks. We show that a minimalist binary routing decision—

selecting strictly between local and global scopes—can eliminate over 90% of full attention operations while maintaining model performance. Our method does not focus on the memory complexity of attention. A recent comprehensive analysis of KV cache eviction and assignment methods in long-context settings is provided in (LI et al., 2025). Their finding shows that sub- $O(n)$ methods struggle with complex interactions.

4.2 Conditional Computation

The notion of selectively skipping computation in deep neural networks can be traced back to conditional computation, which aims to reduce computational cost by activating only a subset of model components conditioned on the input (Bengio et al., 2013). This idea has been extensively explored through mechanisms such as adaptive computation time (Graves, 2017), mixture-of-experts (Shazeer et al., 2017), and mixture-of-depth (Raposo et al., 2024). In Transformer-based architectures, prior work has shown that many attention heads are redundant or contribute unevenly across inputs, motivating approaches such as head pruning and head importance estimation (Michel et al., 2019; Voita et al., 2019). Complementary to these efforts, sparse and block-wise attention mechanisms restrict global attention to a subset of tokens or regions to improve efficiency (Child et al., 2019b; Beltagy et al., 2020b; Zaheer et al., 2020). More recent methods focus on how to make attention more efficient, e.g., FlashAttention (Dao et al., 2022) and hierarchical attention (Yang et al., 2016; Yuan et al., 2025). Orthogonal to these studies, we demonstrate that in most cases global attention is not required at all, providing strong motivation for learning when and where to attend globally.

5 Conclusion

In this paper, we introduced All-or-Here Attention, a simple yet effective paradigm that dynamically toggles between efficient local sliding window attention and full global attention at the token level. Our empirical results on OLMo-2 confirm that the vast majority of global attention operations are redundant. We demonstrate that with a window size of 128 tokens, AHA can eliminate over 88% of full attention without compromising performance on standard benchmarks. Furthermore, our analysis reveals that context dependency follows a long-tail distribution: as the local receptive field expands,

the need for global access decays rapidly. These findings indicate that the key to efficiency lies not only in speeding up full attention, but also in accurately identifying the sparse moments when global context is truly required. We hope this work inspires further exploration of simple, dynamic strategies for efficient language modeling and their GPU-friendly adaptation.

6 Limitations

While our experiments demonstrate the efficacy of AHA in dynamically bypassing redundant attention operations, we identify two primary limitations in our current study.

First, the incremental sparsity gains achieved by our method on long-context tasks are relatively modest. Due to computational resource constraints, our long-context evaluation relies on a strict parameter-efficient fine-tuning setup where only the newly initialized gating parameters are updated. Furthermore, this tuning phase utilizes a limited corpus of long-context data (fewer than 1B tokens). This restricted optimization likely constrains the model’s capacity to fully internalize complex, context-dependent routing patterns. Training the model from scratch or integrating the mechanism during the pre-training phase could potentially unlock a higher sparsity ceiling.

Second, although AHA successfully accelerates inference computation, it does not reduce memory consumption. Because the routing decisions are made dynamically on a per-token basis, an attention head that opts for local context at the current decoding step may still require global access for future tokens. Consequently, the model must preserve the full KV cache in memory during inference. However, this limitation can be largely mitigated by exploiting AHA’s inherent composability. For instance, when integrated with DuoAttention, the combined architecture inherits almost the same KV cache footprint of the static baseline while providing supplementary latency speedups.

References

- Joshua Ainslie, Tao Lei, Michiel de Jong, and 1 others. 2023. CoLT5: Faster long-range transformers with conditional computation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*.
- Jacob Austin, Augustus Odena, Maxwell Nye, and 1 others. 2021. Program synthesis with large language models. *ArXiv*.
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural machine translation by jointly learning to align and translate. *CoRR*.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, and 1 others. 2024. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*.
- Ankur Bapna, Naveen Arivazhagan, and Orhan Firat. 2020. Controlling computation versus quality for neural sequence models. *CoRR*.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020a. Longformer: The long-document transformer. *ArXiv*.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020b. Longformer: The long-document transformer. *arXiv:2004.05150*.
- Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. 2015. Conditional computation in neural networks for faster models. *CoRR*.
- Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *ArXiv*.
- Adithya Bhaskar, Alexander Wettig, Tianyu Gao, and 1 others. 2025. Cache me if you can: How many kvs do you need for effective long-context lms? *CoRR*.
- Tom B. Brown, Benjamin Mann, Nick Ryder, and 1 others. 2020. Language models are few-shot learners. *ArXiv*.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019a. Generating long sequences with sparse transformers. *ArXiv*.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019b. [Generating long sequences with sparse transformers](#). *Preprint*, arXiv:1904.10509.
- Karl Cobbe, Vineet Kosaraju, Mo Bavarian, and 1 others. 2021. Training verifiers to solve math word problems. *ArXiv*.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: fast and memory-efficient exact attention with io-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics*.
- Alexander R. Fabbri, Irene Li, Tianwei She, and 1 others. 2019. [Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model](#). In *Annual Meeting of the Association for Computational Linguistics*.
- Lu Fan and Yue Jiang. 2023. Probability distribution of dependency distance and dependency type in translational language. *Humanities and Social Sciences Communications*.
- Tianyu Fu, Haofeng Huang, Xuefei Ning, Genghan Zhang, Boju Chen, Tianqi Wu, Hongyi Wang, Zixiao Huang, Shiyao Li, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. 2025. Mixture of attention spans: Optimizing LLM inference efficiency with heterogeneous sliding-window lengths. In *Second Conference on Language Modeling*.
- Richard Futrell, Kyle Mahowald, and Edward Gibson. 2015. Large-scale evidence of dependency length minimization in 37 languages. *Proceedings of the National Academy of Sciences*.
- Leo Gao, Jonathan Tow, and 1 others. 2023. [A framework for few-shot language model evaluation](#).
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *CoRR*.
- Alex Graves. 2017. [Adaptive computation time for recurrent neural networks](#). *Preprint*, arXiv:1603.08983.
- Albert Gu and Tri Dao. 2024. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Francois Fleuret. 2020. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*.

677	Wojciech Kryściński, Nazneen Rajani, Divyansh Agarwal, Caiming Xiong, and Dragomir Radev. 2022. Booksum: A collection of datasets for long-form narrative summarization. In <i>Findings of the association for computational linguistics: EMNLP 2022</i> .	Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In <i>Neural Information Processing Systems</i> .	730
678			731
679			732
680			733
681			
682	Nathan Lambert, Jacob Daniel Morrison, Valentina Pyatkin, and 1 others. 2024. TULU 3: Pushing frontiers in open language model post-training. <i>ArXiv</i> .	Elena Voita, David Talbot, Fedor Moiseev, Rico Senrich, and Ivan Titov. 2019. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In <i>Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics</i> , pages 5797–5808.	734
683			735
684			736
685	YUCHENG LI, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. 2025. SCBench: A KV cache-centric analysis of long-context methods. In <i>The Thirteenth International Conference on Learning Representations</i> .		737
686			738
687			739
688		Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, and 1 others. 2025. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. In <i>The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025</i> .	740
689			741
690			742
691	Haitao Liu, Chunshan Xu, and Junying Liang. 2017. Dependency distance: A new perspective on syntactic patterns in natural languages. <i>Physics of life reviews</i> .		743
692			744
693		An Yang, Anfeng Li, Baosong Yang, and 1 others. 2025a. Qwen3 technical report. <i>ArXiv</i> .	745
694	Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? In <i>Advances in Neural Information Processing Systems</i> , volume 32.		746
695			747
696		Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. 2025b. Lserve: Efficient long-sequence llm serving with unified sparse attention. <i>Proceedings of Machine Learning and Systems</i> .	748
697			749
698	Thanh-Tung Nguyen, Xuan-Phi Nguyen, Shafiq R. Joty, and Xiaoli Li. 2020. Differentiable window for dynamic local attention. <i>ArXiv</i> .		750
699			751
700		Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024. Gated delta networks: Improving mamba2 with delta rule. <i>ArXiv</i> .	752
701	Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, and 1 others. 2024. 2 OLMo 2 furious. <i>ArXiv</i> .		753
702			754
703	Long Ouyang, Jeff Wu, Xu Jiang, and 1 others. 2022. Training language models to follow instructions with human feedback. <i>ArXiv</i> .		755
704		Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard H. Hovy. 2016. Hierarchical attention networks for document classification. In <i>North American Chapter of the Association for Computational Linguistics</i> .	756
705			757
706	David Raposo, Samuel Ritter, Blake A. Richards, Timothy P. Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-Depths: Dynamically allocating compute in transformer-based language models. <i>CoRR</i> .		758
707		Jingyang Yuan, Huazuo Gao, Damai Dai, and 1 others. 2025. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In <i>Annual Meeting of the Association for Computational Linguistics</i> .	759
708			760
709		Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big bird: transformers for longer sequences. In <i>Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20, Red Hook, NY, USA. Curran Associates Inc.</i>	761
710			762
711	Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer . <i>Preprint</i> , arXiv:1701.06538.		763
712			764
713			765
714			766
715			767
716	Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. 2019. Adaptive attention span in transformers. <i>ArXiv</i> .		768
717			769
718			770
719	Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A question answering challenge targeting commonsense knowledge. <i>ArXiv</i> .		771
720		Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? In <i>Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers</i> .	772
721			773
722			774
723	Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context llm inference. In <i>ICML</i> .		775
724			776
725		Xiaofeng Zhang, Yikang Shen, Zeyu Huang, and 1 others. 2022. Mixture of attention heads: Selecting attention heads per token. In <i>Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022</i> .	777
726			778
727	Hugo Touvron, Thibaut Lavril, Gautier Izacard, and 1 others. 2023. LLaMA: Open and efficient foundation language models. <i>ArXiv</i> .		779
728			780
729			781